17 · 存储选型决策（ADR-017）：PG、TiDB、TDengine 怎么分

项	内容
文档编号	DT-PdM-17
类型	架构决策记录（ADR-017）
触发问题	"集群里不是已经有 PG 了吗？不够用能不能也存 TiDB？"
结论	三分法：TDengine 存时序、PG16 存事务、TiDB+TiFlash 存分析读模型
日期	2026-10-21

---

1. 先把"有没有 PG"查清楚（实盘）

实例	位置	版本	归属	可否复用
`db-postgres-0`	`dify-plus/db-postgres:5432`	PostgreSQL 15.12（Alpine）	Dify 应用自带	⚠ 技术上能连，工程上不建议
`dataagent-course-postgres`	`dataagent-course:5432`	PG（课程环境）	教学课程	✗ 不可用于生产
`cockroachdb-a/b/c`	`infra`，30Gi ×3	CockroachDB	共享基础设施	⚠ 分布式 SQL，可用但语义差异大

所以答案是：有 PG，但没有"共享 PG 服务"。

这两者的差别不是措辞，是风险：

风险	说明
生命周期耦合	Dify 升级、迁移、重建库时，PdM 生产数据跟着遭殃
备份策略冲突	Dify 的备份窗口与保留策略由它自己定，不会为 PdM 调整
连接数争抢	Dify 有 pgbouncer，PdM 若直连会打乱它的连接池规划
权限边界模糊	共用实例意味着 DBA 权限交叉，审计上说不清
版本被锁定	Dify 停在 PG15，PdM 想用 PG16 的某些特性就无解

> 一条工程常识：数据库是最不该"顺手复用"的东西。
> 复用中间件（缓存、消息、向量库）收益大、耦合小；复用数据库收益小、耦合大。

---

2. 那能不能全放 TiDB？（认真评估，不是敷衍）

TiDB 实盘情况：

项	实测
版本	v8.5.8（协议 8.0.11）
组件	tidb ×1、tikv ×3（各 10Gi）、tiflash ×1（10Gi）、pd ×3（各 5Gi）
连接数	60

项	实测
稳定性	tikv-1 重启 34 次、tikv-2 重启 88 次；Last State = Terminated，Exit Code 137（SIGKILL，典型 OOM）
扩容	local-path StorageClass ALLOWVOLUMEEXPANSION = false，不能在线扩容

2.1 TiDB 能做的（优势）

优势	对本项目的价值
MySQL 协议兼容	Java(JDBC)、Go(go-sql-driver)、Python(PyMySQL) 全通，多语言无障碍
HTAP（TiKV 行存 + TiFlash 列存）	一份数据同时支撑点查与跨基地聚合，这正是 CQRS 读侧的理想形态
水平扩展	20 基地数据量增长时可加 TiKV 节点
强一致（Raft）	方案卡状态、工单回执等关键状态可靠
已在跑	零部署成本

2.2 TiDB 不能做的（硬约束）

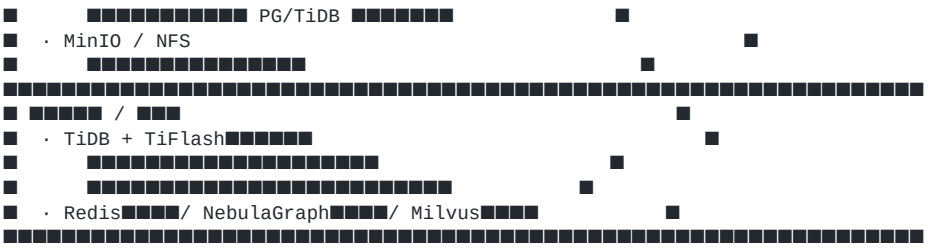
约束	影响	严重度
没有 Row Level Security（RLS）	多租户隔离必须靠应用层 + 视图 + 强制查询构造器，安全等级下降一档	高
容量只有 30Gi（3×10Gi）+ TiFlash 10Gi	20 基地 3 年业务数据 + 读模型会撑爆	高
OOM 历史（Exit 137）	生产主库放上去，等于把命交给一个正在抖的集群	高
local-path 不可扩容	要扩容只能重建 PVC 迁数据	中
JSON 能力弱于 PG	我们的 `contributions`、`evidence` 大量用 JSON；TiDB JSON 无 GIN 索引	中
小事务延迟高于单机 PG	方案卡状态机高频小事务，TiDB 的 Raft 提交成本更高	低

2.3 结论

> TiDB 适合"读"，不适合"写"。把它定位成分析读模型 + 跨基地对标，是扬长避短；把它当主库，是拿稳定性换便利。

---

3. 最终方案：存储三分法



3.1 一句话记忆

问题	答案
要强一致 + 多租户硬隔离？	PG16
要每秒几千点写入 + 按时间扫？	TDengine
要跨 20 基地做聚合分析？	TiDB + TiFlash
要存波形文件？	MinIO / NFS

---

4. 成本对比（三种方案）

方案	新增组件	首期成本	风险	评价
A. 全复用 TiDB	0	最低	OOM + 容量 + 无 RLS	省了小钱，赔了大钱
B. 复用 Dify 的 PG	0	低	生命周期耦合、权限交叉	⚠ 只可做 POC 过渡
C. PG16 新建 + TiDB 复用 + TDengine 新建	2	中	可控	推荐
D. 全新建（PG + TDengine + CH）	3	高	重复建设	浪费存量

方案 C 的额外好处：TiDB 只承担读，即使它再 OOM，业务主流程不受影响，只是看板降级。这符合架构原则 P3 降级优于崩溃。

---

5. 多租户隔离的替代实现（因为 TiDB 无 RLS）

既然主库是 PG，RLS 可用；但读模型在 TiDB，需要补偿措施：

层	措施	强度
身份	JWT 携带 `tenant_id`，网关校验	强
查询构造	所有查询强制经过 `TenantQueryBuilder`，禁止裸 SQL	强
数据	读模型表首列固定 `tenant_id`，建复合索引	中
校验	每条返回结果做租户断言（不一致即抛异常并告警）	强
测试	多租户越权测试纳入每次 CI（构造跨租户查询必须失败）	强

层	措施	强度
审计	所有跨租户访问记安全事件	中

> 不能因为"没有 RLS"就降低隔离要求，只能用更多层补偿。 这是选 TiDB 的隐性成本，必须写进设计。

6. 落地步骤（Ph1 内完成）

步	内容	产出
1	在 `pdm-prod` 部署 PG16 主从（Helm），启用 RLS	可用连接串
2	建 `pdm` 数据库、租户角色、RLS 策略	DDL 脚本
3	部署 TDengine（单机起步，PV 放 worker1）	可用连接串
4	在 TiDB 建 `pdm_read` 库，只授读权限给投影器	读模型 DDL
5	TiKV/TiFlash 资源调优（提高 memory limit，消除 OOM）	调优记录
6	申请 TiKV 扩容（若对标数据量增长）	扩容单

> 第 5 步不是"顺手"，是前置条件：TiDB 现在这个状态，任何读模型压测都可能把它打挂。

7. 需要你确认的补充决策

#	决策	选项	建议
D23	PG 承载方式	① 复用 Dify 实例建独立库（零新增） ② 新建专用 PG16 实例	② 新建（Ph1 就建，避免后期迁移）
D24	TiDB 是否先修	① 先调资源再上读模型 ② 直接上，坏了再说	① 先修（OOM 是已知事实）
D25	波形存储	① 只留边缘 ② 中心留 7 天热数据 ③ 全量中心	① + ② 摘要

> 这三条与原 D1 – D22 一起，构成完整决策集。确认后进入 Ph1 实施。

8. 决策更新（2026-10-21，委托方拍板）

委托方决定：D23 = 全用 TiDB（关系型主库不用 PG）。

本 ADR 的 §3 三分法因此被修订为二分法 + 补偿层：

存储	最终定位
TiDB（TiKV 行存 + TiFlash 列存）	事务主库 + 读模型 + 跨基地分析（全包）
TDengine	时序特征值（TiDB 不擅此道，保留）
MinIO / NFS	波形、报告、模型权重
Redis / Milvus / Nebula	缓存、向量、图谱（复用）

8.1 决策被接受的前提（必须同时满足，否则回退 PG）

#	前提	验收方式
P1	消除 TiKV OOM（Exit 137）	连续 7 天 0 重启
P2	TiKV 扩容至 ≥ 100Gi/节点	`SHOW TABLE ... REGIONS` + 容量核对
P3	多租户补偿层上线（无 RLS）	CI 越权测试 100% 拦截
P4	JSON 高频字段转生成列 + 索引	报告查询 P95 ≤ 200 ms
P5	BR 全备 + 日志备份，RPO ≤ 5 min	恢复演练通过

> 若 P1 或 P2 在 Ph1 内无法达成，本决策自动回退为"PG16 主库 + TiDB 只读"。这条写进 ADR，避免后面扯皮。

8.2 无 RLS 的隔离实现（补偿方案）

```
-- ■■■■■■■■■■ tenant_id■■■■■■■■■■
CREATE TABLE diagnosis (
  tenant_id   VARCHAR(32) NOT NULL,
  diag_id     VARCHAR(40) NOT NULL,
  ...
  PRIMARY KEY (tenant_id, diag_id),
  KEY idx_tenant_asset_time (tenant_id, asset_id, created_at)
);

# ■■■■■■■■■■■■ SQL
class TenantQueryBuilder:
    def __init__(self, tenant_id: str): ...
    def select(self, table: str) -> "TenantQueryBuilder":
        # ■■■■■ WHERE tenant_id = :tid■■■■■■■■■■■
        ...
    def execute(self) -> list[dict]:
        rows = self._run()
        assert all(r["tenant_id"] == self.tenant_id for r in rows), "■■■■■■■■"
        return rows
```

三层防线：构造器强制注入 → 结果断言 → CI 越权用例。缺一层就等于没有隔离。